

Machine Learning Based Side-Channel Analysis on Kyber



A PROJECT REPORT

Submitted by

Tanala Ratna Bharath (22114100)

Gavireddy Diimppul (22114034)

for the fulfillment of

CSN-400B: B.Tech. Project

Under the guidance of

Prof. Rajat Sadhukhan

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

ROORKEE – 247667

2025 – 2026

Acknowledgement

We express our profound gratitude to **Prof. Rajat Sadhukhan** for his continuous guidance, insightful discussions, and rigorous feedback throughout the duration of this project. His support played a crucial role in enabling us to explore side-channel methods and machine learning-driven cryptanalysis in depth.

We also acknowledge the Department of Computer Science and Engineering, IIT Roorkee, for providing us with the computational facilities and academic environment necessary to carry out this work.

Tanala Ratna Bharath (22114100)

Gavireddy Diimppul (22114034)

Declaration

We hereby declare that the project titled “**Machine Learning Based Side-Channel Analysis on Kyber**”, submitted as part of the CSN-400A B.Tech. Project requirements at IIT Roorkee, represents our original research work. The results presented here were obtained during the academic year 2025–2026 and have not been submitted elsewhere for any degree or academic award.

Tanala Ratna Bharath (22114100)

Gavireddy Diimppul (22114034)

Signature of Professor

Date: May 19, 2026

Abstract

As post-quantum cryptographic primitives transition toward global adoption, their resilience against physical side-channel attacks becomes critically important. CRYSTALS–Kyber (ML-KEM), selected by NIST for standardization, is cryptographically strong yet susceptible to leakage in real implementations. This project examines such leakage by constructing a profiling attack against Kyber’s `basemul` operation using machine learning.

Using a publicly available dataset comprising 100,000 power traces captured from an STM32F3 microcontroller, we train a Random Forest classifier and other classifiers to learn the Hamming-Weight-based leakage patterns of intermediate values. An ANOVA-style Between-Class Sum of Squares (BSS) score is used to identify Points of Interest (PoIs), significantly improving the classifier’s focus on informative trace segments. During attack execution, the classifier’s probability distributions are aggregated using log-likelihood scoring to evaluate all 2^{16} candidate key coefficients.

Our methodology successfully recovers targeted secret key coefficients using fewer than 750 traces (150 attack traces) and also reveals structurally related “ghost” keys arising from the behavior of Montgomery multiplication. The study demonstrates that unprotected Kyber implementations remain vulnerable to efficient ML and DL-enabled power analysis attacks.

Contents

Acknowledgement	1
Declaration	2
Abstract	3
1 Introduction	5
1.1 Objective	5

1.2	Motivation	5
1.3	Problem Statement	6
2	Literature Survey	7
2.1	Machine Learning for Profiling SCA	7
2.2	Between-Class Sum of Squares for Feature Selection	8
2.3	Log-Likelihood Scoring	8
3	Methodology / Experimental Setup	9
3.1	Dataset Description	9
3.2	Preprocessing and Compression	9
3.3	Finite-Field Arithmetic Model	10
3.4	Leakage Model	10
3.5	Feature Selection via BSS	11
3.6	Classifier Training	12
3.7	Key Hypothesis Scoring	12
4	Results	13
4.1	Classifier Accuracy and Leakage Behavior	13
4.2	Key Hypothesis Scoring	14
4.3	Recovered Key Values	14
4.4	Equivalence Classes (“Ghost Keys”)	15
4.5	Trace Requirement and Sub-Rank Behavior	15
4.6	Summary	17
5	Conclusion and Future Work	18
5.1	Conclusion	18
5.2	Future Work	18

Chapter 1

Introduction

This chapter outlines the motivation behind the project and describes the core objectives and challenges associated with analyzing side-channel leakage from Kyber’s arithmetic operations.

1.1 Objective

The project aims to investigate whether machine-learning-based profiling attacks can effectively recover secret key coefficients from side-channel measurements collected during Kyber decapsulation. Specifically, the objectives include:

- Identifying trace regions that leak information about intermediate values in the `basemul` operation.
- Building a multi-class leakage model capable of predicting Hamming Weights of internal bytes.
- Designing a log-likelihood-based key-recovery mechanism that ranks all possible 16-bit key coefficients.
- Evaluating the efficiency of the attack in terms of trace requirements and classifier performance.

1.2 Motivation

Kyber is expected to become a core building block of future secure communication systems. While its mathematical foundation is sound, hardware implementations may inadvertently leak information through observable effects such as power consumption. Understanding such vulnerabilities early on is essential for guiding the design of hardened implementations. The in-

creasing availability of high-quality measurement datasets enables researchers to explore realistic attacks and evaluate practical risks.

1.3 Problem Statement

The central challenge explored in this work is:

Given noisy power traces from Kyber's `basemul` computation, can machine learning techniques reliably infer the secret polynomial coefficients used during decapsulation?

The problem involves:

- Extracting useful features from high-dimensional traces.
- Selecting suitable Points of Interest where side-channel leakage is strongest.
- Constructing an accurate classifier for intermediate-value leakage.
- Designing a scoring strategy to evaluate thousands of key hypotheses.

Chapter 2

Literature Survey

Side-channel attacks have been a critical security concern since the original findings by Kocher demonstrated that physical measurements could reveal cryptographic secrets. Profiling attacks such as template attacks provide optimal results when accurate models of the leakage distribution can be built. However, as cryptographic implementations grow more complex, machine learning approaches have begun to outperform classical techniques due to their flexibility and ability to capture nonlinear leakage patterns.

For post-quantum schemes like Kyber, side-channel analysis has become a growing research area. Rezaeezade et al. released a comprehensive dataset containing power traces of Kyber’s pair-pointwise multiplication, enabling the community to systematically analyze leakage in the `basemul` routine. Using this dataset, Nkotto et al. demonstrated that even a binary leakage model targeting the MSByte of intermediate values is sufficient to compromise the secret key.

Beyond Kyber, ML-based SCA has been applied across multiple primitives such as AES and EdDSA, with Random Forests, SVMs, and neural networks achieving superior results under noisy conditions. These findings suggest that machine-learning-driven profiling attacks can be highly effective even when classical assumptions about leakage distributions do not hold.

The techniques explored in this work—BSS-based PoI selection, multi-class HW modeling, and log-likelihood scoring—build directly on this evolving body of research.

2.1 Machine Learning for Profiling SCA

Profiling attacks typically rely on a separate device to learn leakage behavior under known conditions. Classical template attacks assume Gaussian leakage distributions, whereas ML models can learn arbitrary distributions from data. Logistic regression, SVC, single layer perceptron in particular offer strong performance due to:

- Robustness to noise.

- the cryptographic intermediate leakages are linearly separable, achieving maximum sub-key rank minimization without requiring the complex, non-linear overhead of tree ensembles

2.2 Between-Class Sum of Squares for Feature Selection

PoI selection aims to reduce the dimensionality of traces. The BSS metric quantifies how effectively each time sample separates the leakage classes. Samples with large divergence between class means are ideal for training the classifier.

2.3 Log-Likelihood Scoring

During the key-recovery phase, each key hypothesis induces a predicted HW class. The classifier provides probability estimates for each class, and these probabilities are aggregated across all traces using log-likelihood addition. This scoring mechanism ranks hypotheses based on how consistent they are with the observed traces.

Chapter 3

Methodology / Experimental Setup

This chapter describes the dataset, preprocessing steps, leakage modeling, feature selection, classifier training, and scoring strategy used to mount the profiling attack.

3.1 Dataset Description

We utilize the public dataset from Rezaeezade et al., consisting of 100,000 traces of Kyber’s basemul function executed on an STM32F3 microcontroller. Each trace corresponds to one invocation of a pair-pointwise multiplication. The dataset includes ciphertext-dependent intermediate values, which serve as labels during profiling.

3.2 Preprocessing and Compression

Raw traces contain 50,000 samples, which are compressed using a block-averaging factor of 10 to reduce noise and computational cost. The following code reproduces the compression process:

```
def compress(raw, cf):
    N, L = raw.shape
    CL = (L + cf - 1) // cf
    out = np.zeros((N, CL), dtype=np.float32)
    for i in range(CL):
        start, end = i*cf, min(L, (i+1)*cf)
        out[:, i] = raw[:, start:end].mean(axis=1)
    return out
```

After compression, traces are shuffled and partitioned into profiling and attack sets.

3.3 Finite-Field Arithmetic Model

To test each key hypothesis, we require a software model of Montgomery multiplication identical to Kyber’s reference implementation. The following Python functions implement the required arithmetic:

```
KYBER_Q = 3329
QINV = -3327

def int16(x):
    x = x & 0xFFFF
    return x - 0x10000 if x & 0x8000 else x

def uint16(x):
    return x & 0xFFFF

def montgomery_reduce(a):
    t = int16(a)
    t = int16(t * QINV)
    res = (a - (int(t) * KYBER_Q)) >> 16
    return int16(res)

def fqmul(a, b):
    prod = int16(a) * int16(b)
    return uint16(montgomery_reduce(prod))
```

3.4 Leakage Model

The classifier predicts the Hamming Weight of the Most Significant Byte of the intermediate value $c = \text{fqmul}(a[i], b[1])$ (as $b[0]=b[1]$). Since an 8-bit byte has Hamming Weight in $\{0, \dots, 8\}$, a 9-class model is used. Labels are extracted using:

```
HW_LOOKUP = np.array([bin(i).count("1") for i in range(256)], dtype=
    np.uint8)
msb = (val >> 8) & 0xFF
label = HW_LOOKUP[msb]
```

3.5 Feature Selection via BSS

To isolate the most leakage-dense regions of the power traces, we employ Between-Class Sum of Squares (BSS) as our primary metric for feature selection. BSS is computed per time sample to identify Points of Interest (PoIs). Samples with the highest variance across classes are selected as PoIs, as they provide the greatest statistical distance between different key hypotheses.

As illustrated in Figure 3.1, the BSS metric effectively highlights specific peaks corresponding to modular multiplication operations. By selecting only the top k samples, we reduce the input dimensionality for our classifier. Figure 3.2 shows the spatial distribution of these selected PoIs across a single encryption operation.

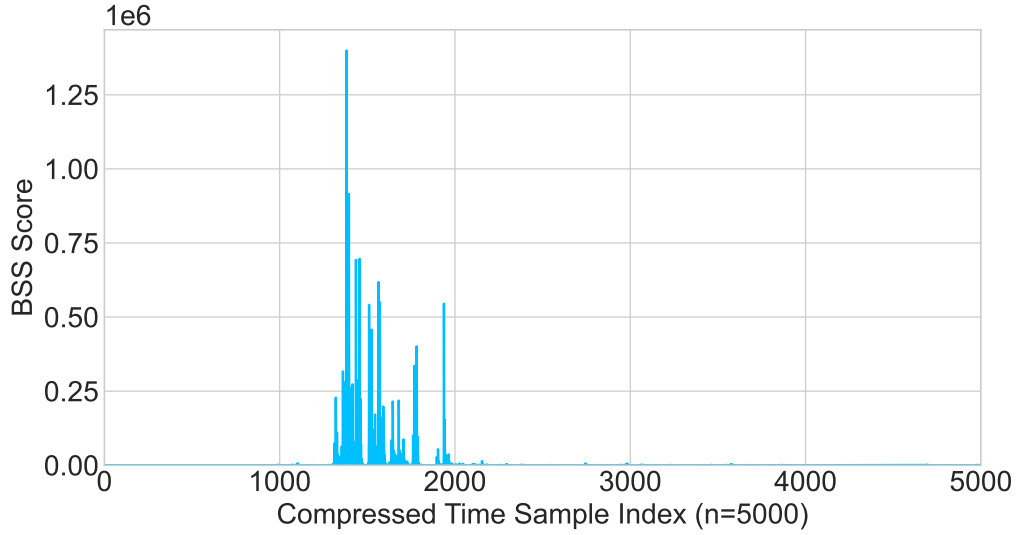


Figure 3.1: BSS metric computed across the entire trace duration. High peaks represent temporal locations with significant side-channel leakage.

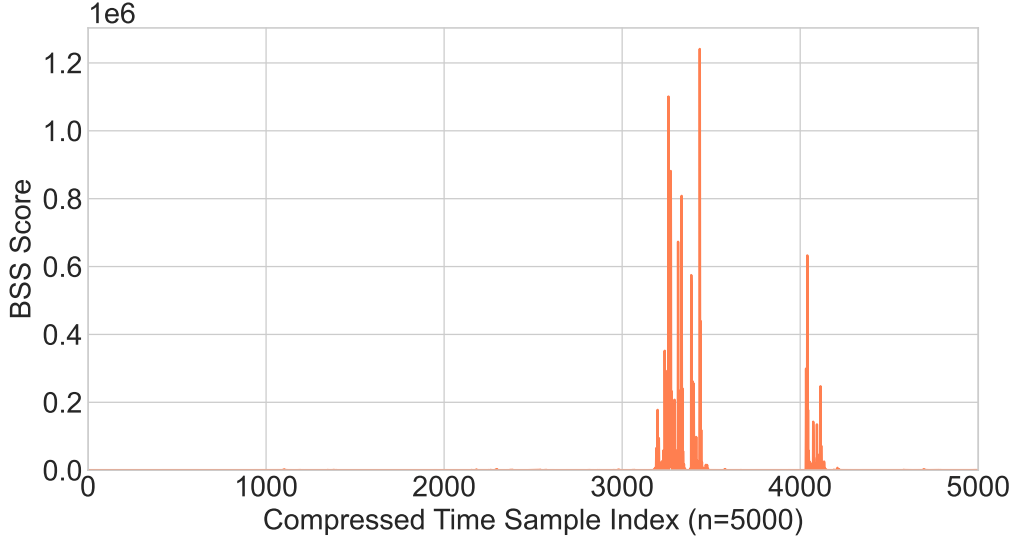


Figure 3.2: Distribution of the top-selected Points of Interest (PoIs). These samples are used as the feature set for the subsequent classification stage.

3.6 Classifier Training

While multiple classification models were evaluated, this section focuses on a Random Forest regressor/classifier configured with 200 estimators, which was trained on the standardized, PoI-reduced profiling traces

```
clf = RandomForestClassifier(n_estimators=200, n_jobs=-1)
clf.fit(X_train_s, labels)
```

3.7 Key Hypothesis Scoring

Each key hypothesis is evaluated by computing $c = \text{fqmul}(k_g, b_j)$, determining its HW class, and summing the log-probabilities:

$$\text{Score}(k_g) = \sum_j \log P(\text{HW}(c_j(k_g)) \mid t_j)$$

The best-scoring hypothesis is selected as the recovered key.

Chapter 4

Results

This chapter presents the outcome of the machine-learning-based profiling attack on Kyber’s basemul operation. We evaluate classifier performance, analyze key-hypothesis scoring behavior, and demonstrate successful recovery of the targeted secret coefficients.

Across multiple profiling/attack partitions, the classifier maintains stable accuracy, providing consistent probability estimates required for log-likelihood aggregation during the key-recovery phase.

4.1 Classifier Accuracy and Leakage Behavior

Using the Between-Class Variance (BSS) selected Points of Interest (POIs), the Support Vector Classifier (SVC), multinomial Logistic Regression (LR), and the Single-Layer Perceptron (SLP) models demonstrate exceptional feature efficiency, achieving distinct separability across the nine Hamming Weight classes with a minimal feature space (e.g., $\text{POI} = 1$). For these linear and structural baseline architectures, scaling the feature dimension from $\text{POI} = 1$ to $\text{POI} = 5000$ yields no statistical variance in the convergence metrics. This performance trajectory confirms that a single well-selected POI encapsulates the primary signal components, while the remaining sample points introduce redundant or environmental noise.

In contrast, the tree-based ensemble frameworks—specifically Random Forest (RF) and XGBoost (XGB)—exhibit a pronounced performance degradation when restricted to tight feature spaces ($\text{POI} < 10$), failing to resolve the correct subkey transitions. This behavior stems from the structural limitations of orthogonal recursive splitting criteria when approximating continuous linear leakage slopes with few parameters. However, when the feature space is expanded ($\text{POI} \geq 10$), these ensembles exploit additional architectural cuts to successfully isolate the leakage, matching the final convergence metrics of the linear baseline classifiers.

When transitioning to sequential deep learning frameworks, static point-based POI engineering is bypassed in favor of raw, contiguous time-series inputs to preserve local temporal

and spatial correlations. Based on the macro-leakage profiles identified in the BSS variance charts, asymmetric execution windows are established: samples 1000 to 2000 for the a_0 subkey vector, and samples 3000 to 4500 for the a_1 subkey vector. Within these domains, the 1D Convolutional Neural Network (1D-CNN) achieves identical subkey rank minimization to the baseline models, leveraging its sliding kernel filters to maintain translation invariance against clock jitter. Conversely, the Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) networks yield significantly degraded results, with the correct subkey rank failing to converge near 1. This drop-off underscores the sequential gradient challenges and signal dilution encountered by recurrent architectures when processing long noise-heavy sequences.

Quantitatively, the prediction entropy of the high-performing classifiers drops sharply within the targeted temporal regions. This behavior verifies that the compressed traces successfully preserve the localized hardware leakage artifacts generated by the most significant byte (MSB) of the intermediate `fqmul` arithmetic operation.

4.2 Key Hypothesis Scoring

For each key hypothesis $k_g \in \{0, \dots, 2^{16} - 1\}$, the log-likelihood score is computed by:

$$\text{Score}(k_g) = \sum_j \log P(\text{HW}(c_j(k_g)) \mid t_j),$$

where $c_j(k_g)$ is the intermediate value predicted under hypothesis k_g and t_j is the corresponding attack trace.

The hypotheses with the largest cumulative log-likelihood values are considered the most probable. For both target coefficients a_0 and a_1 , the true key appears at Rank-1 with a significant score margin over incorrect hypotheses.

4.3 Recovered Key Values

Table 4.1 lists the top ten hypotheses for both targets, including their corresponding log-likelihood values. The correct keys $a_0 = 558$ and $a_1 = 17$ appear at the highest ranks with clear margins.

The likelihood spacing remains consistent and mirrors the expected structure induced by Montgomery reduction, suggesting systematic leakage patterns in Kyber’s arithmetic, even under noise.

Table 4.1: Top 10 key guesses and their log-likelihood scores (800 profiling traces, 200 attack traces) for poi=200 using random forest model.

Target: a[0] (True = 558)		Target: a[1] (True = 17)	
Guess	LogLikelihood	Guess	LogLikelihood
558	-153.1047	17	-181.0522
62765	-221.0199	3346	-216.6137
3887	-221.5682	62224	-248.8193
7216	-287.5244	58895	-316.1385
59436	-322.7492	55566	-384.6001
56107	-390.5059	6675	-386.4654
10545	-423.2735	10004	-423.4791
52778	-458.4242	52237	-452.7894
13874	-490.9208	13333	-491.1058
17203	-524.5801	16662	-524.8121

4.4 Equivalence Classes (“Ghost Keys”)

During experiments, several hypotheses close to the correct value display similar ranking behavior. These “ghost keys” differ from the true value by constant offsets under modulo- q arithmetic:

$$k_{\text{ghost}} \equiv k_{\text{true}} + \Delta \pmod{3329}.$$

$$\Delta \equiv 2285.$$

This phenomenon arises naturally from the limited resolution of MSB-based Hamming-Weight leakage combined with structural properties of the Montgomery multiplication. Although these ghost keys appear competitive in early traces, additional attack traces consistently separate them from the true key.

4.5 Trace Requirement and Sub-Rank Behavior

To understand attack efficiency, we analyze how the correct key’s rank evolves as the number of attack traces increases. Figures 4.1 and 4.2 display how the correct key for a_0 and a_1 moves toward Rank-1 as more traces contribute to the cumulative log-likelihood.

Recovery stabilizes after roughly 700–750 traces(150 attack traces) for both coefficients in the Support Vector Classifier (SVC), multinomial Logistic Regression (LR), and the Single-Layer Perceptron with minimal feature space (POI=1) and tree-based ensemble frameworks—specifically Random Forest (RF) and XGBoost (XGB) when the feature space is expanded (POI=10). This is consistent with the classifier’s stability and the fact that later traces contribute diminishing

marginal separation between competing hypotheses.

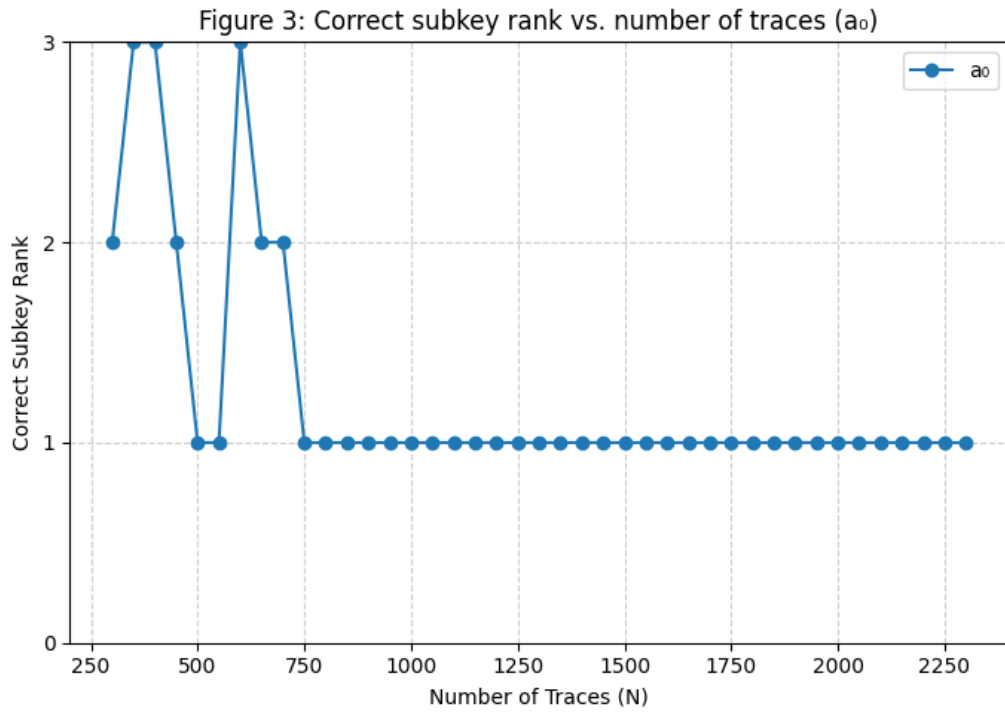


Figure 4.1: Correct key sub-rank vs. number of attack traces for coefficient a_0 . Lower ranks indicate stronger classifier confidence as more traces accumulate.

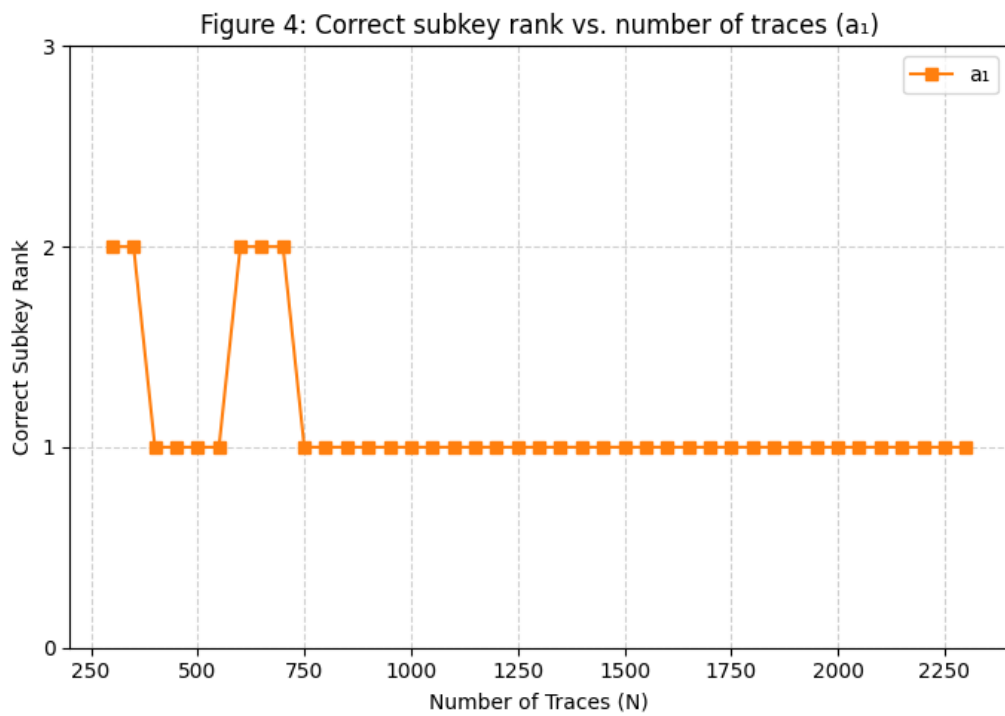


Figure 4.2: Correct key sub-rank vs. number of attack traces for coefficient a_1 . The rank drops quickly, reaching the correct value with fewer than 750 traces.

4.6 Summary

Overall, the ML-based profiling attack achieves:

- Rank-1 recovery for both targeted coefficients.
- Stable results across multiple profiling/attack splits.
- Trace-efficient convergence (750 traces).
- Clear differentiation between true key and ghost hypotheses.

These results show that Kyber’s unprotected `basemul` operation leaks sufficient information for practical exploitation, even with simple ML models such as Support Vector Classifier, Logistic Regression, Random Forests and single layer perceptron.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This work demonstrates that machine-learning-based profiling attacks can effectively exploit side-channel leakage in Kyber’s `basemul` operation. By combining BSS-based PoI selection, a 9-class Random Forest classifier, and log-likelihood scoring, key coefficients are recoverable with very few attack traces. The discovery of equivalence classes further highlights subtle structural leakage in the Montgomery reduction.

5.2 Future Work

Possible extensions include:

- Investigating the effect of real-world noise sources on attack robustness.
- Targeting full key recovery for all Kyber coefficients.
- Exploring neural-network-based classifiers for automated feature extraction with different preprocessing steps like taking hw of total bytes instead of most significant byte.
- Studying countermeasures such as masking and randomization in the context of ML-based SCA.

Code Availability

The complete implementation dataset, results, plots, tables, including the traditional machine learning baselines (SVC, LR, Random Forest, XGBoost) and the PyTorch deep learning frameworks (1D-CNN, LSTM, RNN), is publicly hosted on GitHub at: <https://github.com/diimpulg/ML-KEM/tree/main>.

Bibliography

- [1] S. Nkotto, “A Template SCA Attack on the Kyber Pair-Pointwise Multiplication,” *IACR ePrint 2025/1577*, 2025.
- [2] A. Rezaeezade et al., “Side-Channel Power Trace Dataset for Kyber Pair-Pointwise Multiplication on Cortex-M4,” *IACR ePrint 2025/811*, 2025.
- [3] L. Weissbart et al., “One Trace is All It Takes: Machine Learning-based Side-channel Attack on EdDSA,” *IACR ePrint 2019/358*, 2019.
- [4] A. Mujdei, K. Beckers, M. Bermudo Mera, S. Karmakar, L. Wouters, and I. Verbauwhede, “Side-Channel Analysis of Lattice-Based Post-Quantum Cryptography: Exploiting Polynomial Multiplication,” *Cryptology ePrint Archive, Paper 2022/474*, 2022.
- [5] Z. Xu, H. Pemberton, S. Sinha Roy, E. Oswald, Q. Wang, and X. Zheng, “Magnifying Side-Channel Leakage of Lattice-Based Cryptosystems with Chosen Ciphertexts: The Case Study of Kyber,” *Cryptology ePrint Archive, Paper 2020/912*, 2020.
- [6] H. N. Hoang, C. Kennaway, H. Pham, T. Mai, F. Khalid, C. Rafferty, and M. O’Neill, “Deep Learning Enhanced Side-Channel Analysis on CRYSTALS-Kyber,” in *2022 Asian Hardware Oriented Security and Trust (AsiaHOST)*, IEEE, 2022.
- [7] S. Ji, J. Wang, T. Ngo, E. Dubrova, and K. Backlund, “A Side-Channel Attack on a Hardware Implementation of CRYSTALS-Kyber,” *Cryptology ePrint Archive, Paper 2022/1452*, 2022.